AM Modulation & Demodulation

Laboratory Manual

Using MATLAB and the ADALM-PlutoSDR

Experiment Overview

This manual covers the end-to-end implementation of Amplitude Modulation (AM/DSB-LC) using MATLAB and the ADALM-PlutoSDR software-defined radio platform.

Department of Electrical Engineering Department

June 4, 2026

Contents

1 Introduction

Amplitude Modulation (AM) is one of the oldest and most fundamental analog modulation techniques in communication engineering. In AM, the amplitude of a high-frequency *carrier* signal is varied in proportion to the *message* (baseband) signal, while the carrier frequency itself remains constant.

This experiment implements the complete AM modulation and demodulation chain using MATLAB and the ADALM-PlutoSDR (Pluto Software Defined Radio). The Pluto SDR provides a low-cost, full-duplex radio platform operating from 70 MHz to 6 GHz, making it an ideal tool for hands-on RF communications experiments.

1.1 Learning Objectives

Upon completing this experiment, students should be able to:

1. Derive and implement the mathematical model of DSB-LC AM modulation.
2. Generate a complex baseband (IQ) signal suitable for SDR transmission.
3. Transmit and receive an AM signal using the PlutoSDR.
4. Demodulate an AM signal using envelope detection.
5. Design and apply a Butterworth low-pass filter to recover the message.
6. Interpret time-domain and frequency-domain plots of the system.

2 Theoretical Background

2.1 AM (DSB-LC) Signal Model

In Double-Sideband Large Carrier (DSB-LC) AM, the transmitted signal is:

$$s_{\mathrm{AM}}(t) = A_c\big[1 + k_a\, m(t)\big]\cos(2\pi f_c t) \tag{1}$$

where:

- A_c — carrier amplitude (normalized to 1 in this experiment),
- k_a — amplitude sensitivity (modulation index),
- $m(t)$ — normalized message signal $\big(|m(t)| \le 1\big)$,
- f_c — carrier frequency.

For a sinusoidal message $m(t) = \cos(2\pi f_m t)$, Equation 1 expands to:

$$s_{\mathrm{AM}}(t) = A_c\cos(2\pi f_c t) + \frac{k_a A_c}{2}\cos\big[2\pi(f_c + f_m)t\big] + \frac{k_a A_c}{2}\cos\big[2\pi(f_c - f_m)t\big] \tag{2}$$

This reveals three spectral lines: the carrier at f_c and two sidebands at $f_c \pm f_m$.

2.2 Modulation Index

The modulation index μ (here $= k_a$ for a single-tone message) determines the depth of modulation:

$$\mu = k_a = \frac{A_{\max} - A_{\min}}{A_{\max} + A_{\min}} \tag{3}$$

For $\mu < 1$ the signal is under-modulated; $\mu = 1$ gives 100% modulation; $\mu > 1$ causes over-modulation (envelope distortion and impossible envelope detection). This experiment uses $k_a = 0.8$ (80% modulation) to ensure clean envelope detection.

2.3 Baseband (IQ) Representation

The PlutoSDR operates on complex baseband signals. The *analytic signal* (Hilbert transform) of $s_{\mathrm{AM}}(t)$ shifts all energy to positive frequencies and creates the complex IQ representation:

$$s_{\mathrm{BB}}(t) = \mathcal{H}\{s_{\mathrm{AM}}(t)\} = s_{\mathrm{AM}}(t) + j\,\hat{s}_{\mathrm{AM}}(t) \tag{4}$$

where $\hat{s}_{\mathrm{AM}}(t)$ is the Hilbert transform of $s_{\mathrm{AM}}(t)$. The in-phase component $I(t) = \mathrm{Re}\{s_{\mathrm{BB}}\}$ and quadrature component $Q(t) = \mathrm{Im}\{s_{\mathrm{BB}}\}$ are the two physical channels fed to the SDR DAC.

2.4 Envelope Detection

The simplest AM demodulator computes the magnitude of the received IQ signal:

$$e(t) = \left|r_{\mathrm{IQ}}(t)\right| = \sqrt{I_r^2(t) + Q_r^2(t)} \tag{5}$$

After removing the DC offset (proportional to A_c) and normalizing:

$$\hat{m}(t) = \frac{e(t) - \overline{e}}{\max\left|e(t) - \overline{e}\right|} \approx m(t) \tag{6}$$

A low-pass filter is then applied to suppress residual high-frequency components introduced by hardware imperfections or channel noise.

2.5 Butterworth Low-Pass Filter

The recovered envelope is filtered with a second-order Butterworth IIR filter. The Butterworth design maximizes the passband flatness. The transfer function in the z-domain is obtained via the bilinear transform of the analogue prototype. The normalized cutoff frequency is:

$$\Omega_n = \frac{f_{\mathrm{cut}}}{f_s/2}, \qquad f_{\mathrm{cut}} = 8\ \mathrm{kHz} \tag{7}$$

Zero-phase filtering (`filtfilt`) is used to eliminate group-delay distortion.

3 System Parameters

Table 1 summarises all key system parameters used in the experiment.

Table 1: System Parameters

Parameter	Symbol	Value	Unit
Sample rate	f_s	1 000 000	Sps
Tx/Rx centre freq.	f_{RF}	100	MHz
Baseband carrier	f_{c1}	100	kHz
Message frequency	f_m	5	kHz
Modulation index	k_a	0.8	—
Number of samples	N	$1024 \times 16 = 16\,384$	—
Tx gain	—	-20	dBm
Rx gain	—	30	dB
LPF cutoff	f_{cut}	8	kHz
Butterworth order	n	2	—

4 Equipment and Software Requirements

- ADALM-PlutoSDR (connected via USB)
- MATLAB R2020b or later
- **Communications Toolbox** (for `sdrtx`, `sdrrx`)
- **Communications Toolbox Support Package for ADALM-PlutoSDR**
- Signal Processing Toolbox (for `butter`, `filtfilt`, `hilbert`)
- SMA RF cable (loopback Tx → Rx) or antenna pair

5 MATLAB Implementation

5.1 System Block Diagram

Signal Chain

$m(t) \longrightarrow$ **AM Modulator** $\longrightarrow$ **Hilbert** (IQ) $\longrightarrow$ **PlutoSDR Tx** $\xrightarrow{\text{RF}}$ **PlutoSDR Rx** $\longrightarrow$ **Envelope Det.** $\longrightarrow$ **LPF** $\longrightarrow \hat{m}(t)$

Figure 1: End-to-end AM modulation/demodulation system block diagram.

5.2 Complete MATLAB Code

```
%% ===== AM MODULATION / DEMODULATION WITH PLUTO SDR =====
clear; clc; close all;

%% System Parameters
fs   = 1e6;         % Sample rate (1 MSps)
fc   = 100e6;       % RF centre frequency for Tx/Rx (100 MHz)
fc1  = 100e3;       % Baseband carrier frequency (100 kHz)
fm   = 5e3;         % Message frequency (5 kHz)
ka   = 0.8;         % Amplitude sensitivity (modulation index = 0.8)
N    = 1024*16;     % Number of samples
t    = (0:N-1)'/fs;% Time vector (column)

%% Generate Message Signal (single audio tone)
m_t = cos(2*pi*fm*t);           % Normalized message signal

%% AM Modulation (DSB-LC)
s_am = (1 + ka*m_t) .* cos(2*pi*fc1*t);

%% Normalize for PlutoSDR (peak amplitude <= 1)
s_am_norm = s_am / max(abs(s_am)) * 0.9;

%% Convert to complex baseband via Hilbert transform
s_bb = hilbert(s_am_norm);      % Analytic signal (I + jQ)

%% === TRANSMIT ===
tx = sdrtx('Pluto');
tx.CenterFrequency    = fc;
tx.BasebandSampleRate = fs;
tx.Gain               = -20;    % Reduce gain to avoid saturation
transmitRepeat(tx, s_bb);

%% === RECEIVE ===
rx = sdrrx('Pluto');
rx.CenterFrequency    = fc;
rx.BasebandSampleRate = fs;
rx.GainSource         = 'Manual';
rx.Gain               = 30;
rx.SamplesPerFrame    = N;
rx.OutputDataType     = 'double';

[rx_iq, ~, ~] = rx();

%% === DEMODULATION: Envelope Detection ===
env            = abs(rx_iq);           % Envelope = sqrt(I^2 + Q^2)
env_dc_removed = env - mean(env);      % Remove DC (carrier component)

%% Normalize recovered signal
m_rec = env_dc_removed / max(abs(env_dc_removed));

%% Butterworth Low-Pass Filter Design
fc_msg       = 5e3;          % Message tone (Hz)
fc_cut       = 8e3;          % Cutoff frequency (Hz)
butter_order = 2;            % Filter order

Wn        = fc_cut / (fs/2);           % Normalized cutoff
[b_but, a_but] = butter(butter_order, Wn, 'low');

```

```
% Zero-phase filtering (no phase distortion)
m_rec_lp  = filtfilt(b_but, a_but, m_rec);
m_rec_lp  = m_rec_lp / max(abs(m_rec_lp)); % Renormalize

%% === PLOTS ===
figure('Name','AM Modulation Results','NumberTitle','off');

subplot(3,2,1); plot(t(1:500)*1e3, m_t(1:500));
title('Message Signal m(t)');
xlabel('Time (ms)'); ylabel('Amplitude');

subplot(3,2,2); plot(t(1:500)*1e3, real(s_bb(1:500)));
title('AM Signal (Baseband I)'); xlabel('Time (ms)');

subplot(3,2,3); plot(t(1:500)*1e3, real(rx_iq(1:500)));
title('Received IQ Signal'); xlabel('Time (ms)');

subplot(3,2,4); plot(t(1:500)*1e3, m_rec(1:500));
title('Demodulated Signal - Envelope Detection'); xlabel('Time (ms)');

subplot(3,2,5);
NFFT = 4096;
f_ax = (-NFFT/2:NFFT/2-1)*fs/NFFT/1e3;
S_am = 20*log10(abs(fftshift(fft(s_bb,NFFT)))/NFFT);
plot(f_ax, S_am); xlim([-150 150]);
title('AM Baseband Spectrum');
xlabel('Freq (kHz)'); ylabel('Mag (dB)');

subplot(3,2,6);
plot(t(1:500)*1e3, m_rec_lp(1:500), 'b', 'LineWidth', 1.2);
title('Demodulated Signal (LP Filtered)');
xlabel('Time (ms)'); ylabel('Amplitude'); grid on;

release(tx); release(rx);
```

Listing 1: AM Modulation and Demodulation with PlutoSDR

6 Step-by-Step Procedure

Step 1. Hardware Setup. Connect the ADALM-PlutoSDR to the PC via USB. Connect the Tx SMA port to the Rx SMA port using a short coaxial cable (loopback) or use two antennas placed in proximity.

Step 2. Install Support Package. In MATLAB, go to *Add-Ons* → *Communications Toolbox Support Package for ADALM-PlutoSDR* and follow the installer.

Step 3. Verify Connection. Run `sdrinfo('Pluto')` in the MATLAB Command Window and confirm the device is detected.

Step 4. Set Parameters. Open the script and confirm the parameters in Table 1 match your setup. Adjust `tx.Gain` and `rx.Gain` if the received signal is too weak or clipped.

Step 5. Generate and Modulate. The script creates a 5 kHz cosine message $m(t)$ and applies DSB-LC AM modulation with $k_a = 0.8$. The Hilbert transform converts

the real AM signal to a complex baseband (IQ) signal.

Step 6. Transmit. `transmitRepeat` continuously streams s_{BB} through the PlutoSDR Tx at 100 MHz.

Step 7. Receive. The Rx object captures $N = 16\,384$ complex IQ samples at the same centre frequency.

Step 8. Demodulate. Envelope detection via `abs(rx_iq)` extracts the signal envelope. The DC offset is removed by subtracting the mean. A 2nd-order Butterworth LPF (cutoff 8 kHz) is applied using `filtfilt` for zero-phase response.

Step 9. Inspect Results. Six subplots are generated (Section 7). Compare the recovered signal to the original message $m(t)$.

Step 10. Release Hardware. `release(tx)` and `release(rx)` free the SDR resources.

7 Experimental Results and Analysis

Figure 2 shows the six output plots generated by the MATLAB script. Each subplot is discussed below.

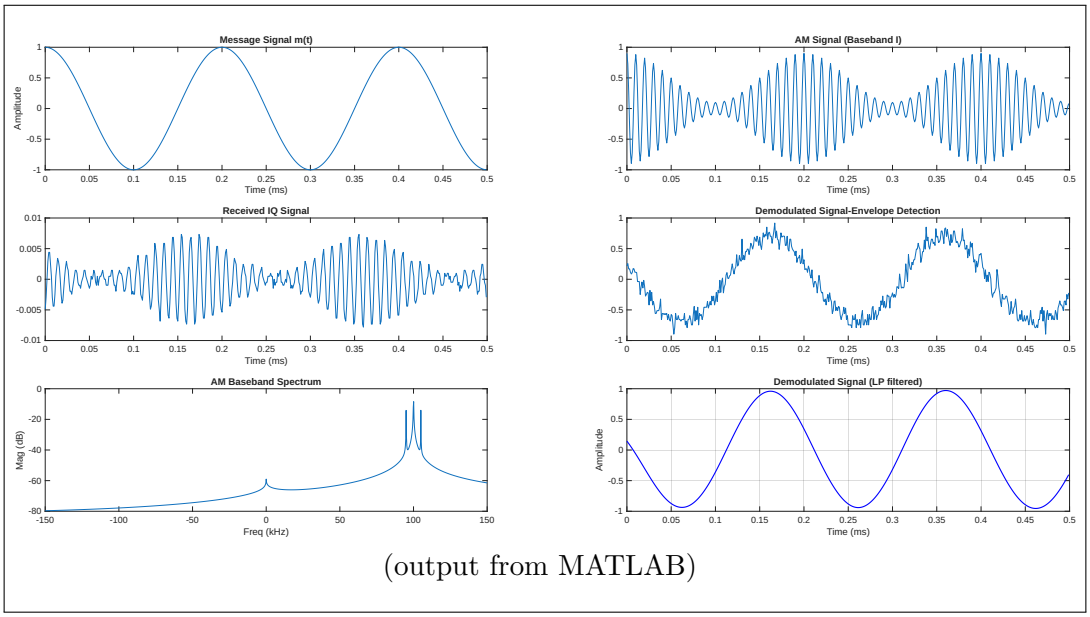

Figure 2: AM modulation and demodulation results (PlutoSDR loopback).

7.1 Message Signal $m(t)$ — Subplot 1

A pure 5 kHz sinusoidal tone of unit amplitude. It serves as the baseband information signal. The time axis spans 0 to 0.5 ms, showing approximately 2.5 cycles of the message.

7.2 AM Baseband I-Component — Subplot 2

This is the real part of the analytic (complex baseband) signal $s_{\mathrm{BB}}(t)$, equivalent to the in-phase (I) channel. The sinusoidal amplitude envelope varying at $f_m = 5$ kHz is clearly visible on the $f_{c1} = 100$ kHz carrier, confirming correct DSB-LC modulation with $k_a = 0.8$.

7.3 Received IQ Signal — Subplot 3

The signal captured by the PlutoSDR receiver. The amplitude is significantly lower ($\approx \pm 0.01$) compared to the transmitted signal, which is expected due to path loss, the Tx gain of -20 dBm, and the hardware RF chain attenuation. The waveform shape (AM envelope) is nonetheless preserved.

7.4 Envelope-Detected Signal — Subplot 4

After computing $e(t) = |r_{\text{IQ}}(t)|$ and removing the DC offset, the recovered waveform closely resembles the original message $m(t)$. Minor distortion and noise are visible, attributable to receiver noise and hardware non-idealities.

7.5 AM Baseband Spectrum — Subplot 5

The power spectral density (in dB) of s_{BB} is plotted over ± 150 kHz. Three dominant spectral lines are visible:

- **Centre (0 kHz):** residual DC / carrier component.
- ± 100 **kHz:** the baseband carrier at f_{c1}.
- ± 95 **and** ± 105 **kHz:** the two AM sidebands at $f_{c1} \pm f_m$.

7.6 Low-Pass Filtered Demodulated Signal — Subplot 6

Applying the 2nd-order Butterworth LPF (cutoff 8 kHz) with `filtfilt` removes residual carrier harmonics and noise while preserving the 5 kHz message. The resulting waveform is a clean sinusoid closely matching $m(t)$, validating the complete AM demodulation chain.

8 Key Signal Processing Concepts

Hilbert Transform and Analytic Signal

The `hilbert()` function in MATLAB returns the *analytic signal*, not simply the Hilbert transform. The analytic signal $x_a(t) = x(t) + j\hat{x}(t)$ has a one-sided spectrum (energy only at positive frequencies), which is the correct format for SDR baseband transmission.

Zero-Phase Filtering with `filtfilt`

Standard IIR filtering introduces a frequency-dependent group delay. `filtfilt` applies the filter *twice* — once forward and once backward — resulting in zero net phase shift. This prevents time-domain distortion of the recovered waveform.

Gain Selection

- **Tx Gain** $= -20$ **dBm:** prevents PlutoSDR DAC saturation and reduces self-interference in loopback.

- **Rx Gain = 30 dB:** amplifies the weak received signal to a usable level. If clipping occurs, reduce this value.

9 Observations and Discussion Questions

Answer the following questions in your lab report:

Q1. Why is the received signal amplitude (Subplot 3) much smaller than the transmitted signal (Subplot 2)? What hardware and propagation factors contribute?

Q2. What would happen to Subplot 4 if the modulation index were increased to $k_a = 1.2$? Sketch the expected envelope shape and explain.

Q3. Explain why `filtfilt` is preferred over a single `filter` call for this application.

Q4. In the spectrum (Subplot 5), identify the carrier and sidebands. What is the bandwidth of the transmitted AM signal? How does it relate to the message bandwidth?

Q5. How would you modify the code to transmit a speech signal (audio file) instead of a single sinusoidal tone? List the changes required.

Q6. If the cutoff frequency of the Butterworth filter were lowered from 8 kHz to 3 kHz, how would Subplot 6 change?

10 Troubleshooting

Table 2: Common Issues and Solutions

Problem	Solution
PlutoSDR not detected	Run `sdrinfo('Pluto')` and check USB connection. Reinstall the support package driver.
Received signal is all zeros	Verify Tx/Rx are tuned to the same `CenterFrequency`. Increase `rx.Gain`.
Received signal is clipped / saturated	Reduce `rx.Gain` or `tx.Gain`.
Demodulated signal is noisy	Increase `N` (more samples), lower Butterworth cutoff, or improve antenna placement.
`filtfilt` instability	Ensure Wn $\in (0, 1)$. Raise the filter cutoff or reduce the order.
Phase inversion in $\hat{m}(t)$	Multiply `m_rec` by -1 if the recovered signal is inverted (normal for some channel phases).

11 Conclusion

This experiment demonstrated a complete AM (DSB-LC) communication link using MATLAB and the ADALM-PlutoSDR. Key outcomes include:

- Successful generation of a DSB-LC AM signal with modulation index $k_a = 0.8$ and verification of its spectrum showing a carrier and two sidebands.
- Conversion to complex baseband (IQ) format using the Hilbert transform for PlutoSDR transmission at 100 MHz.
- Over-the-air (loopback) reception and envelope detection of the received IQ signal.
- Clean message recovery using DC removal and a 2nd-order Butterworth low-pass filter with zero-phase implementation (`filtfilt`).

The experiment confirms that envelope detection is an effective and simple demodulation method for AM signals with modulation index < 1, and that software-defined radio platforms like the PlutoSDR provide an accessible environment for implementing real RF communication systems.